

Tuesday, June 16, 2026 2:04 PM

<https://blog.xpnsec.com/linux-process-injection-aka-injecting-into-sshd-for-fun/>

Avant fork() :

Process parent
pid = 1234

fork() est appelé...

Après fork() :

Process parent
pid = 1234

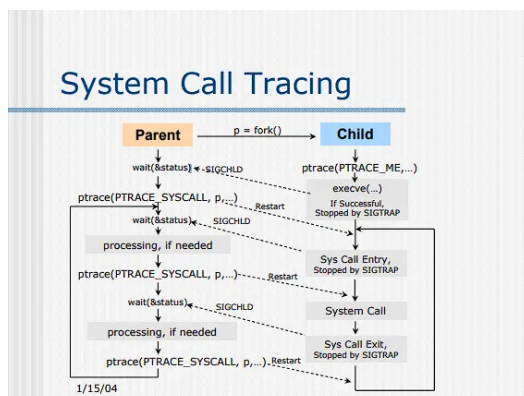
fork() retourne
→ 5678 (PID fils)

Process enfant
pid = 5678

fork() retourne
→ 0

↓

<https://notes.eatonphil.com/2023-10-01-intercepting-and-modifying-linux-system-calls-with-pttrace.html>



Commande	Ce que ça fait
PTRACE_SYSCALL	s'arrêter à chaque syscall
PTRACE_SINGLESTEP	exécuter instruction par instruction
PTRACE_GETREGS	lire tous les registres
PTRACE_SETREGS	modifier les registres
PTRACE_peekTEXT	lire la mémoire
PTRACE_POKETEXT	écrire dans la mémoire
PTRACE_ATTACH	s'attacher à un processus déjà lancé

You can use the `PTRACE_SYSCALL` request: it restarts the child process !!!
Donc a utiliser apres une utilisation et quand le child c'est eteint.

Bon lançons notre programme :

```

adelachot@adelachot-20251204-prtacej$ gdb ch3.bin
GNU gdb (CentOS Stream) 16.3-3.el9
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ch3.bin...
(No debugging symbols found in ch3.bin)
(gdb) r AAAAAAAAAAAAAA
Starting program: /home/adelachot/Desktop/shellcode_project/cracking/ctf/rootme/ptrace/ch3.bin AAAAAAAAAAAAAA

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.centos.org/>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.

To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x77ffc000
Debugger detected ... Exit
[Inferior 1 (process 3949)] exited with code 0[]
(gdb)

```

J'ai vu aussi qu'on pouvait aussi détourné ce anti debug par soit patcher (evidemment) la fonction ptrace ou alors utiliser une sorte d'emulation permanente pour faire croire au programme tracé qu'il est tracé par son propre traceur et pas le mien.

Protection anti-reverse engineering

Première protection anti-debugging ?
Un processus ne peut-être tracé que par **un seul** debugger à la fois.
→ Auto-trapage pour empêcher tout attachement ultérieur

Réponse des analystes
Émulation de l'appel à `ptrace()` pour qu'il n'échoue jamais.

Réponse à la réponse des analystes
Lancement de deux processus qui se traquent mutuellement avec un dialogue permanent pour s'assurer de la survie de l'autre.

Bon le temps que j'arrive à comprendre ce qu'il se passe...

Voici déjà mon code pour l'instant :

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<sys/ptrace.h>
#include<unistd.h>
#include<sys/wait.h>
#include <sys/user.h>

int main(int argc, char ** argv){

    int child = 0;
    pid_t Cpid = 0;
    char * args[] = {"ch3.bin", "ALICE", NULL};
    char * command = "./ch3.bin";
    int status = 0;
    int syscall_count = 0;
    int entering = 1; //code d'entrée(1)/sortie(2)
    child = fork();

    struct user_regs_struct regs; //structure pour gérer/lire les registres

    if(child < 0){
        printf("erreur dans la creation de l'enfant\n");
        return 1;
    }
    //-----//

    if(child == 0){
        Cpid = getpid(); //PID de l'enfant depuis l'enfant lui meme donc a utiliser que dans l'enfant
        printf("enfant process n° %d crée\n", Cpid);
        printf("Demande a l'enfant de le tracer.\n");
        ptrace(PTRACE_TRACEME, 0, NULL, NULL);
        printf("Execution du programme\n");
        execvp(command , args);
        perror("execvp");
        exit(1);
    }
    //-----//

    printf("Je suis le parent, mon fils à le PID %d\n", child);
    waitpid(child, &status, 0);
    printf("enfant stoppé apres execvp\n\n");

    while(1){

        //on relance l'enfant jusqu'au prochain syscall
        if(ptrace(PTRACE_SYSCALL, child, NULL, NULL) == -1){
            perror("SYSCALL");
        }
        waitpid(child, &status, 0);
        //on checke si l'enfant est dead quicouben
        if(WIFEXITED(status)){
            //printf("Enfant terminé. %d syscalls interceptés.\n", syscall_count);
            break;
        }

        //ici rax contient le numéro du syscall
        if(ptrace(PTRACE_GETREGS, child, NULL, &regs) == -1){
            perror("GETREGS");
        }
        continue;
        //si c'est bon on peut continuer l'analyse
        else{
            //printf("J'ai recuperer des infos importantes. \n/r\tSyscall n°%llu à l'adresse : rip=0x%llx\n",
            regs.orig_rax, regs.rip);

            long Peek = ptrace(PTRACE_PEEKTEXT, child, (void *)regs.rip, NULL);
            if(Peek == -1){
                perror("PEEKTEXT");
            }

        }

        else{
            printf("Instruction à rip : 0x%lx\n", regs.rip);
            syscall_count++;
        }

        //utiliser les regs avec entering
        if(entering){
            printf("Entrée syscall n°%llu\n", regs.orig_rax);
        }
        else{
            //ici rax contient le numéro du syscall
            printf("1-Sortie syscall, retour= %ld\n", regs.rax);
        }
    }
}
```

```
//printf("2-Sortie syscall, retour= 0x%lx\n", regs.rax);
if(regs.orig_rax == 26){
printf("VALEUR DE RETOUR : ptrace intercepté ! valeur réelle = %lld\n", regs.rax);
printf("ptrace appelé avec commande=%llu\n", regs.rdi);
//on doit changer la valeur de retour de eax
regs.rax = 0; //on force le changement à zéro
if(ptrace(PTRACE_SETREGS, child, NULL, &regs) == -1){
perror("SETREGS");
}
else {
printf("VALEUR DE RETOUR de rax changé !\n");
}
}
}
entering = !entering;

}
}
return 0;
}
```

Ce qui me donne ce résultat quand je l'utilise contre le binaire :

Avant :

```
Instruction à rip : 0xf7f78589
Entrée syscall n°45
Instruction à rip : 0xf7f78589
1-Sortie syscall, retour= 136740160
Instruction à rip : 0x8048949
Entrée syscall n°243
Instruction à rip : 0x8048949
1-Sortie syscall, retour= 0
Instruction à rip : 0xf7f78589
Entrée syscall n°45
Instruction à rip : 0xf7f78589
1-Sortie syscall, retour= 136875328
Instruction à rip : 0xf7f78589
Entrée syscall n°45
Instruction à rip : 0xf7f78589
1-Sortie syscall, retour= 136876032
Instruction à rip : 0xf7f78589
Entrée syscall n°26
Instruction à rip : 0xf7f78589
1-Sortie syscall, retour= -1
Instruction à rip : 0xf7f78589
Entrée syscall n°197
Instruction à rip : 0xf7f78589
1-Sortie syscall, retour= 0
Instruction à rip : 0xf7f78589
Entrée syscall n°192
Instruction à rip : 0xf7f78589
1-Sortie syscall, retour= 4160163840
Instruction à rip : 0xf7f78589
Entrée syscall n°4
Debugger détecté ... Exit
Instruction à rip : 0xf7f78589
1-Sortie syscall, retour= 27
Instruction à rip : 0xf7f78589
Entrée syscall n°252
```

Après :

```
1-Sortie syscall, retour= -1
VALEUR DE RETOUR : ptrace intercepté ! valeur réelle = -1
ptrace appelé avec commande=134515488
VALEUR DE RETOUR de rax changé !
Instruction à rip : 0xf7f87589
Entrée syscall n°197
Instruction à rip : 0xf7f87589
1-Sortie syscall, retour= 0
Instruction à rip : 0xf7f87589
Entrée syscall n°192
Instruction à rip : 0xf7f87589
1-Sortie syscall, retour= 4160225280
Instruction à rip : 0xf7f87589
Entrée syscall n°4
#####
Instruction à rip : 0xf7f87589
1-Sortie syscall, retour= 61
Instruction à rip : 0xf7f87589
Entrée syscall n°4
## Bienvenue dans ce challenge de cracking ##
Instruction à rip : 0xf7f87589
1-Sortie syscall, retour= 61
Instruction à rip : 0xf7f87589
Entrée syscall n°4
#####
Instruction à rip : 0xf7f87589
1-Sortie syscall, retour= 61
Instruction à rip : 0xf7f87589
Entrée syscall n°4
Instruction à rip : 0xf7f87589
1-Sortie syscall, retour= 1
Instruction à rip : 0xf7f87589
Entrée syscall n°197
Instruction à rip : 0xf7f87589
1-Sortie syscall, retour= 0
Instruction à rip : 0xf7f87589
Entrée syscall n°192
Instruction à rip : 0xf7f87589
```

On a réussi à intercepter ce syscall a ptrace et on l'a remplacé par 0 pour que ce soit le programme par du principe qu'aucun autre programme le trace. Donc ce sera toujours bon.

Adresses :

Password : 0xf7f87589 **ksuiealohgy** ???

Lors de la verification de l'input les jump vers la bad giri :
(peut etre patcher avec ptrace)

08048A5
08048B4
08048C1
08048D0

Jump vers la good giri : 08048E2

Bref je sais que je dois étudier cette routine qui compare notre input avec cette input mais en prenant que certain caractère de cette input : **ksuiealohgy**

```

text:00404497 .noting db 008h ; DATA XREF: main+9Eto
text:00404498 mov dl, [ebp+var_14]
text:00404499 mov eax, [ebp+var_C]
text:0040449A add eax, 4
text:0040449B mov al, [eax]
text:0040449C cmp dl, al
text:0040449D jnz short loc_004044E4 ; bad girl
text:0040449E mov dl, [ebp+var_13]
text:0040449F mov eax, [ebp+var_C]
text:004044A0 add eax, 3
text:004044A1 mov al, [eax]
text:004044A2 cmp dl, al
text:004044A3 jnz short loc_004044E4 ; bad girl
text:004044A4 mov dl, [ebp+var_14]
text:004044A5 mov eax, [ebp+var_C]
text:004044A6 inc eax
text:004044A7 mov al, [eax]
text:004044A8 cmp dl, al
text:004044A9 jnz short loc_004044E4 ; bad girl
text:004044AA mov dl, [ebp+var_C]
text:004044AB add eax, [ebp+var_C]
text:004044AC mov al, [eax]
text:004044AD cmp dl, al
text:004044AE jnz short loc_004044E4 ; bad girl
text:004044AF sub esp, 0Ch
text:004044B0 push offset aGoodPassword ; "nGood password !!!\n"
text:004044B1 call _IO_puts
text:004044B2
text:004044B3

```

A etudier !

Peut etre que le ptrace c'était juste pour bypasser l'anti debug... Mais dans ce cas la il faudrait que je patche vraiment ça.

Ptrace address: 00058a70

```

[adelachatch@adelachatch-20251204 ptrace]$
[adelachatch@adelachatch-20251204 ptrace]$ objdump -d ch3.bin | grep -A10 "ptrace"
0040410: e8 5b 06 01 00      call 8058a70 <ptrace>
0040415: 83 c4 10           add $0x10,%esp
0040418: 85 c0             test %eax,%eax
004041a: 79 1a           jns 8048436 <main+0x46>
004041c: 83 ec 0c           sub $0xc,%esp
004041f: 68 04 20 0c 08     push $0x80c2004
0040424: e8 a7 0e 00 00     call 8049200 <_IO_puts>
0040429: 83 c4 10           add $0x10,%esp
004042c: b8 01 00 00 00     mov $0x1,%eax
0040431: e9 c3 00 00 00     jmp 80484f9 <_notng+0x62>
0040436: 83 ec 0c           sub $0xc,%esp
0058a70 <ptrace>:
0058a70: 55             push %ebp
0058a71: 89 e5           mov %esp,%ebp
0058a73: 83 ec 18           sub $0x18,%esp
0058a76: 8b 5d f9         mov %ebx,%eax
0058a79: 8b 5d f9         mov %ebx,%esi
0058a7c: 8b 5d 08         mov 0x8(%ebp),%ebx
0058a7f: 8d 75 f4         lea -0xc(%ebp),%esi
0058a82: 8d 43 ff         lea -0x1(%ebx),%eax
0058a85: 8b 55 10         mov 0x10(%ebp),%edx
0058a88: 83 f8 02         cmp $0x2,%eax

```

ON va patcher le jmp par un jne a la place :

long adresse = 0x804841a;

En analysant la routine qui gère notre input, on peut remarquer que :

```

.text:00404495 -----
.text:00404497 .notng db 008h ; DATA XREF: main+9Eto
.text:00404498 -----
.text:00404498 mov dl, [ebp+var_16] ; on met notre input dans dl (on met un byte vu que c'est dl) donc on met la première lettre de notre input
.text:00404499 mov eax, [ebp+var_C] ; on met ksuaiealohgy dans eax
.text:0040449E add eax, 4 ; on se deplace de 4 octets donc iealohgy
.text:004044A1 mov al, [eax] ; on met i dans al (iealohgy)
.text:004044A3 cmp dl, al ; on compare notre input avec i
.text:004044A5 jnz short loc_004044E4 ; if not good we jmp to the bad girl
.text:004044A7 mov dl, [ebp+var_15] ; on met la deuxième valeur de notre input dans dl
.text:004044AA mov eax, [ebp+var_C] ; on met ksuaiealohgy dans eax
.text:004044AD add eax, 5 ; on avance de 5 octets donc ealohgy
.text:004044B0 mov al, [eax] ; on met e dans al
.text:004044B2 cmp dl, al ; on compare si pas bon on jmp dans bad girl
.text:004044B4 jnz short loc_004044E4
.text:004044B6 mov dl, [ebp+var_14] ; on met ksuaiealohgy dans eax
.text:004044B9 mov eax, [ebp+var_C] ; on met ksuaiealohgy dans eax
.text:004044BC inc eax ; on avance de 1 octet donc suaiealohgy
.text:004044BD mov al, [eax] ; on met s dans al
.text:004044BF cmp dl, al ; on compare si pas bon on jmp sur bad girl
.text:004044C1 jnz short loc_004044E4
.text:004044C3 mov dl, [ebp+var_13]
.text:004044C6 mov eax, [ebp+var_C] ; on met ksuaiealohgy dans eax
.text:004044C9 add eax, 0Ah ; on avance de 10 octets : gy
.text:004044CC mov al, [eax] ; on met g dans al
.text:004044CE cmp dl, al ; on compare si pas bon on jmp sur bad girl
.text:004044D0 jnz short loc_004044E4
.text:004044D2 sub esp, 0Ch
.text:004044D5 push offset aGoodPassword ; "nGood password !!!\n"
.text:004044DA call _IO_puts
.text:004044DF add esp, 10h
.text:004044E2 jmp short loc_004044F4
.text:004044E4 -----
.text:004044E4 .text:004044E4 loc_004044E4: ; CODE XREF: main+B5:j
.text:004044E4 sub esp, 0Ch ; main+C4:j ...

```

Et donc on trouve : easy...

Voici mon code final avec ptrace :

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<sys/ptrace.h>
#include<unistd.h>
#include<sys/wait.h>
#include<sys/user.h>

int main(int argc, char **argv){

int child = 0;
pid_t Cpid = 0;
char * args[] = {"ch3.bin", "ksuaiealohgy", NULL};
char * command = "./ch3.bin";
int status = 0;
int syscall_count = 0;
int entering = 1; //code d'entrée(1)/sortie(2)
int patched_already = 0; //pour patcher une seule fois
child = fork();

if(child < 0){
printf("erreur dans la creation de l'enfant\n");
return 1;
}

//-----

if(child == 0){
Cpid = getpid(); //PID de l'enfant depuis l'enfant lui meme donc a utiliser que dans l'enfant
printf("enfant process n° %d crée\n", Cpid);
printf("Demande a l'enfant de le tracer.\n");
ptrace(PTRACE_TRACEME, 0, NULL, NULL);
printf("Execution du programme\n");

```

```
execvp(command , args);
perror("execvp");
exit(1);
}
//-----//

printf("Je suis le parent, mon fils à le PID %d\n", child);
waitpid(child, &status, 0);
printf("enfant stoppé apres execvp\n\n");

while(1){

//on relance l'enfant jusqu'au prochain syscall
if(pttrace[PTRACE_SYSCALL, child, NULL, NULL] == -1){
perror("SYSCALL");
}
waitpid(child, &status, 0);
//on checke si l'enfant est dead quicoubeh
if(WIFEXITED(status)){
//printf("Enfant terminé. %d syscalls interceptés.\n", syscall_count);
break;
}

//ici rax contient le numéro du syscall
if(pttrace[PTRACE_GETREGS, child, NULL, &regs] == -1){
perror("GETREGS");
continue;
}
//-----//

//utiliser les regs avec entering
if(entering){
printf("Entrée syscall n°%llu\n", regs.orig_rax);
}

else{
//ici rax contient le numéro du syscall
printf("1-Sortie syscall, retour= %ld\n", regs.rax);
//printf("2-Sortie syscall, retour= 0x%lx\n", regs.rax);

//-----//
IMPORTANT:-----//

if(regs.orig_rax == 26){
printf("VALEUR DE RETOUR : ptrace intercepté ! valeur réelle = %ld\n", regs.rax);
printf("ptrace appelé avec commande=%llu\n", regs.rdi);
//on doit changer la valeur de retour de eax
regs.rax = 0; //on force le changement à zéro
if(pttrace[PTRACE_SETREGS, child, NULL, &regs] == -1){ //on force rax = 0
perror("SETREGS");
}

if(!patched_already){ //if not patched already
long adresse = 0x804841a;
//on récupère la valeur a cette adresse 0x804841a
long original = ptrace(PTRACE_PEEKTEXT, child, (void*)adresse, NULL);
printf("avant patch : 0x%lx\n", original);
long patched = (original & 0xfffffffffff0) | 0xb;
if(pttrace[PTRACE_POKETEXT, child, (void*)adresse, (void*)patched] == -1){
perror("POKETEXT");
}

else {
printf("jns -> jmp patché à 0x%lx\n", adresse);
printf("VALEUR DE RETOUR de rax changé \n");
patched_already = 1;
}
}
}

syscall_count++;
entering = !entering;

}

return 0;

}
```

REPONSE DES AUTRES PERSONNES DE ROOTME:

Sachant le fonctionnement de la protection, essayons de la contourner proprement, et de récupérer dynamiquement le mot de passe, via GDB.

En l'état, le binaire détecte la présence de GDB :

```
gdb ./ch3.bin
run
Starting program: ch31/root-me/ch3.bin
Debugger detected ... Exit
[Inferior 1 (process 3444)] exited with code 01
```

En y regardant de plus près, le résultat de la fonction "prace" n'est pas intelligemment utilisé par le programme. En effet, le binaire détermine s'il est débuggé uniquement en fonction du code retour de "ptrace" :

```
0x8048410:    call    0x805b70 (ptrace)
0x8048415:    add     esp,0x10
0x8048418:    test    eax,eax
0x804841a:    jns     0x8048436 (not debug)
```

Eax contient le code retour de ptrace, et l'instruction "test eax, eax" vérifie que le registre soit égal à zéro. Autrement dit, le programme vérifie que ptrace retourne bien zéro.

Or, d'après la documentation ptrace, lorsqu'un dbugger est détecté, le code retour de la fonction est "-1".

Pour déboguer paisiblement ce programme, il suffit alors d'indiquer à gdb de modifier le code retour de ptrace en le passant systématiquement à zéro :

```
gdb
gdb-peda$ catch syscall ptrace
Catchpoint 1 (syscall 'ptrace' [26])
gdb-peda$ commands i
Type commands for breakpoint(s) i, one per line.
End with a line saying just "end".
>set ($eax) = 0
>cont
>end
```

Il est maintenant possible d'explorer convenablement la suite du binaire au travers de GDB.

Pour trouver le mot de passe, rien de très compliqué, puisqu'une comparaison est faite caractère par caractère sur la chaîne passée au programme.

Les comparaisons sont toujours du type :

```
0x80484bf:    cmp     di,al
```

Toutefois, "cmp" compare les caractères selon l'encodage de la communication aux données.

Les arguments de commande de l'invite de commande sont interprétés en ASCII.

Il suffit alors d'en observer le contenu avec la commande :

```
gdb-peda$ p $al  
$16 = 0x73
```

Armée d'une table ASCII, les quatre caractères "0x65", "0x61", "0x73" et "0x79" deviennent le mot de passe : "****".

C'est très drôle parce que je me casse la tête pour un challenge aussi facile mais je suis très contente d'avoir appris à coder un debugger avec ptrace !

